

✓ ***THE DEFENSE OF WESTEROS***

The winter is coming, and the Night's Watch is preparing their defense against the more than possible attack from the White Walkers, a one hundred thousand man army that lurks in the darkness. To prevent it, Lord Commander has entrusted us with analyzing images that our ravens get from beyond the wall. The goal is to analyze these images to find and detect enemies and possible threats by implementing histograms, look-up-tables and different techniques for image processing that we are going to explain further in this notebook.



```
try:
    from google.colab import drive
    drive.mount('/content/drive')
    # If your notebook and images are in your Drive, change directory accordingly:
    %cd '/content/drive/Othercomputers/Mi portátil/cuadernos_extra'
except ImportError:
    pass

import numpy as np
from scipy import signal
import cv2
import matplotlib.pyplot as plt
import matplotlib
from ipywidgets import interactive, fixed, widgets
matplotlib.rcParams['figure.figsize'] = (15.0, 15.0)
import random

images_path = './images/'
```

```
Mounted at /content/drive
/content/drive/Othercomputers/Mi portátil/cuadernos_extra
```



CHAPTER I: THE NIGHT AT WINTERFELL

For this first part, we are going to inspect the images from our ravens and try to improve the global contrast in order to make our job easier. This will involve analyzing the pixel distribution through histograms to understand the current state of the image and then applying techniques to spread out these pixel intensities, enhancing visibility of critical details that might otherwise be hidden in the low-contrast blizzard conditions.

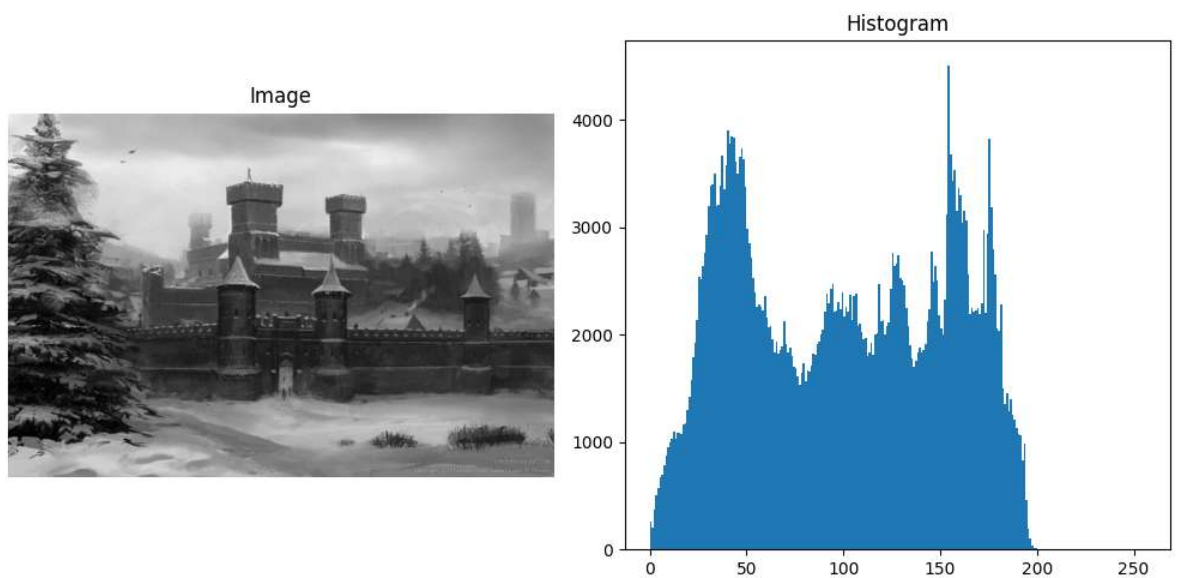
```
image = cv2.imread(images_path + 'winterfell_blizzard.jpg', cv2.IMREAD_GRAYSCALE)

plt.figure(figsize=(10, 5)) # Increased figure size
plt.subplots_adjust(wspace=0.3) # Add spacing between subplots

plt.subplot(1,2,1)
plt.title("Image")
plt.imshow(image, cmap='gray')
plt.axis('off') # Hide axes for cleaner image display

# Now, show its histogram
plt.subplot(1,2,2)
plt.title("Histogram")
plt.hist(image.ravel(), 256, range=[0, 256])

plt.tight_layout() # Adjust layout to prevent overlapping
plt.show()
```



By seeing the histogram of our picture, we can determine that the image has a big concentration of pixels between 0 and 200, but there are no pixels higher than 200. This indicates a low-contrast image, especially in the brighter regions, where details might be lost due to the compressed range of pixel intensities. To effectively identify threats and features in these challenging conditions, we need to enhance the contrast, which histogram equalization can help achieve by distributing the pixel values across the full dynamic range.

✓ HISTOGRAM EQUALIZATION

After the night watch of the ravens, they bring us completely dark images, where, in addition, the snow and fog make it difficult for us to detect enemies. For this, we are going to use histogram equalization to differentiate them. As we can see, in dark images, all pixels are concentrated in the range [0,30] of our histogram, making it impossible to see anything in the dark. With histogram equalization, we will make the pixel intensities more evenly distributed across the histogram, and thus be able to distinguish the silhouettes of the white walkers.

```
import cv2
import matplotlib.pyplot as plt
```

```
import numpy as np

def equalize_chart(image, verbose=False):
    """ Applies histogram equalization to the luminance channel of an image in YCr

    Args:
        image (numpy.ndarray): Input image in BGR color space.
        verbose (bool): If True, plots the original and processed images along

    Returns:
        numpy.ndarray: Equalized image in BGR color space.
    """
    # 1. Transformación al espacio de color YCrCb desde BGR
    ycrcb_image = cv2.cvtColor(image, cv2.COLOR_BGR2YCrCb)

    # 2. Ecualización del canal de luminancia (Y, canal 0)
    ycrcb_image[:, :, 0] = cv2.equalizeHist(ycrcb_image[:, :, 0])

    # 3. Reconversión al espacio de color BGR original para el retorno
    out_image = cv2.cvtColor(ycrcb_image, cv2.COLOR_YCrCb2BGR)

    # 4. Bloque de visualización (separado del procesamiento principal)
    if verbose:
        plt.figure(figsize=(10, 8))

        # Conversión a RGB exclusivamente para una visualización correcta en Matplotlib
        image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
        out_image_rgb = cv2.cvtColor(out_image, cv2.COLOR_BGR2RGB)

        # Muestra de la imagen original
        plt.subplot(2, 2, 1)
        plt.imshow(image_rgb)
        plt.title('Original image')
        plt.axis('off')

        # Muestra de la imagen procesada
        plt.subplot(2, 2, 2)
        plt.imshow(out_image_rgb)
        plt.title('Equalized histogram image')
        plt.axis('off')

        # Histograma original (usando el canal Y de luminancia)
        orig_y = cv2.cvtColor(image, cv2.COLOR_BGR2YCrCb)[:, :, 0]
        plt.subplot(2, 2, 3)
        plt.hist(orig_y.ravel(), bins=256, range=[0, 256])
        plt.title('Original histogram (Y channel)')

        # Histograma ecualizado
        plt.subplot(2, 2, 4)
        plt.hist(ycrcb_image[:, :, 0].ravel(), bins=256, range=[0, 256])
        plt.title('Equalized histogram (Y channel)')

        plt.tight_layout()
        plt.show()

    return out_image
```

```
# Read image
image = cv2.imread(images_path + 'batalla.jpg',-1)

# Equalize its histogram
interactive(equalize_chart, image=fixed(image), verbose=fixed(True))
```

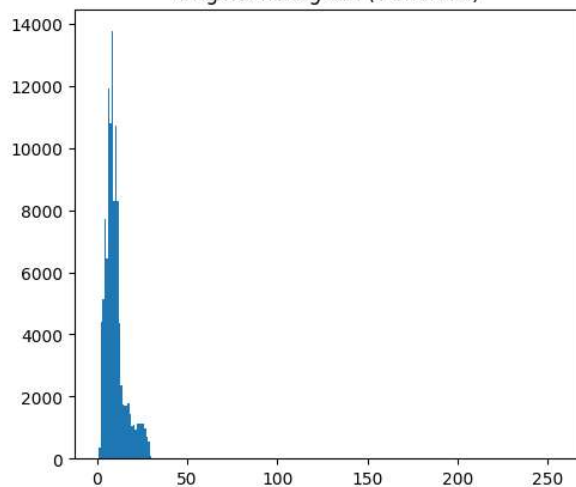
Original image



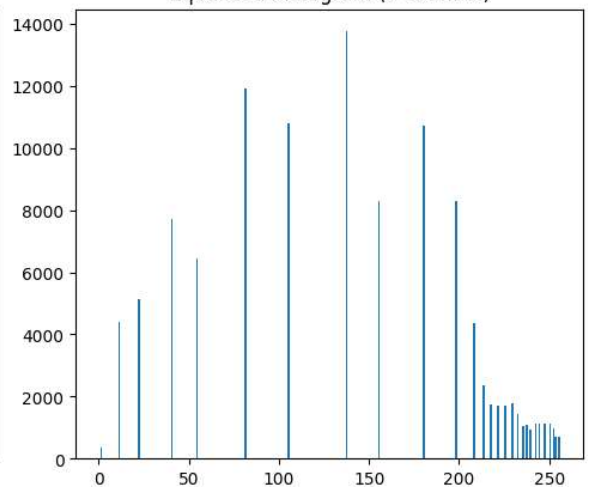
Equalized histogram image



Original histogram (Y channel)



Equalized histogram (Y channel)



Thanks to the incredible power of histogram equalization, we've boosted the contrast, revealing the enemy army in all its glorious gloom. No more squinting into the blizzard, Lord Commander! Those White Walkers thought they were being clever, but our pixel-manipulation prowess has exposed their icy faces. Now we know exactly who's coming for tea (and a battle!), and we can finally tell if that's a direwolf or just a really fluffy snowdrift. Victory is practically in the bag!



✓ SMOOTHING

Even after enhancing contrast with histogram equalization, images can suffer from noise, which obscures important details. To address this, we can apply smoothing techniques like the Gaussian filter. Smoothing helps to reduce random variations in image intensity, making features more discernible without losing too much detail. This initial step of noise reduction can significantly improve the quality of subsequent image processing operations.

```
def gaussian_filter(image, w_kernel, sigma, verbose=False):
    """ Applies Gaussian filter to an image and display it.

    Args:
        image: Input image
        w_kernel: Kernel aperture size
        sigma: standard deviation of Gaussian distribution
        verbose: Only show images if this is True

    Returns:
        smoothed_img: smoothed image
    """
    # Write your code here!

    # Create kernel using separability property
    s = sigma
    w = w_kernel
    kernel_1D = np.float32([1 / (s * np.sqrt(2 * np.pi)) * np.exp(-(z**2) / (2 *
    vertical_kernel = kernel_1D.reshape(2*w+1,1) # Reshape it as a matrix with jus
    horizontal_kernel = kernel_1D.reshape(1,2*w+1) # Reshape it as a matrix with j
    kernel = signal.convolve2d(vertical_kernel, horizontal_kernel) # Get the 2D ke

    # Convolve image and kernel
    smoothed_img = cv2.filter2D(image,-1,kernel)

    if verbose:
```

```
plt.figure(figsize=(10, 5)) # Set a good figure size
# Show the initial image
plt.subplot(121)
plt.imshow(image, cmap='gray')
plt.title('Noisy')
plt.axis("off")

# Show the resultant one
plt.subplot(122)
plt.imshow(smoothed_img, cmap='gray')
plt.title('Gaussian filter')
plt.axis("off")
plt.tight_layout() # Adjust layout to prevent overlapping titles/labels
plt.show() # Display the plots

return smoothed_img
```

```
# Interact with the kernel size and the sigma value
noisy_img = cv2.imread(images_path + 'smoothing_fog.jpeg', 0)
interactive(gaussian_filter, image=fixed(noisy_img), w_kernel=(0,5,1), sigma=(1,3,
```

w_kernel 2
sigma 1.90

Noisy



Gaussian filter



✓ CLAHE

After addressing general image noise with smoothing, we can now refine the contrast enhancement using more advanced techniques. While global histogram equalization can introduce artifacts, Contrast Limited Adaptive Histogram Equalization (CLAHE) offers a more localized approach. CLAHE divides the image into small regions, or 'tiles,' and performs equalization on each one, limiting the contrast to avoid over-amplification of noise. This method provides a better quality picture by enhancing

contrast adaptively, making details clearer, especially in areas that might still be too dark or too bright.

```
import cv2
import matplotlib.pyplot as plt
import numpy as np
import ipywidgets as widgets
from ipywidgets import interactive, fixed

def clahe_chart(image, clip_limit=2.0, tile_grid_size=(8,8), verbose=False):
    """ Applies CLAHE to an image and shows the result.

    Args:
        image: Input image
        clip_limit: Threshold for contrast limiting
        tile_grid_size: Size of grid for histogram equalization
        verbose: Only show images if this is True

    Returns:
        out_image: CLAHE enhanced image
    """

    # Transform image to YCrCb color space
    image = cv2.cvtColor(image, cv2.COLOR_BGR2YCrCb)
    out_image = np.copy(image)

    # Create CLAHE object
    clahe = cv2.createCLAHE(clipLimit=clip_limit, tileGridSize=tile_grid_size)

    # Apply CLAHE to first band of the YCrCb image
    out_image[:, :, 0] = clahe.apply(out_image[:, :, 0])

    if verbose:
        plt.figure(figsize=(15, 10)) # Increased figure size for better visibility

        # Apply global histogram equalization for comparison
        equalized_image = np.copy(image)
        equalized_image[:, :, 0] = cv2.equalizeHist(equalized_image[:, :, 0])

        # Plot histograms
        plt.subplot(2,3,4)
        plt.hist(image[:, :, 0].ravel(), 256, range=[0, 256])
        plt.title('Original histogram')

        plt.subplot(2,3,5)
        plt.hist(equalized_image[:, :, 0].ravel(), 256, range=[0, 256])
        plt.title('Equalized histogram')

        plt.subplot(2,3,6)
        plt.hist(out_image[:, :, 0].ravel(), 256, range=[0, 256])
        plt.title('CLAHE histogram')

        # Reconvert images to RGB
        image = cv2.cvtColor(image, cv2.COLOR_YCrCb2RGB)
        equalized_image = cv2.cvtColor(equalized_image, cv2.COLOR_YCrCb2RGB)
        out_image = cv2.cvtColor(out_image, cv2.COLOR_YCrCb2RGB)
```

```
# Show the images
plt.subplot(2,3,1)
plt.imshow(image)
plt.title('Original Image')

plt.subplot(2,3,2)
plt.imshow(equalized_image)
plt.title('Global equalization')

plt.subplot(2,3,3)
plt.imshow(out_image)
plt.title('CLAHE')

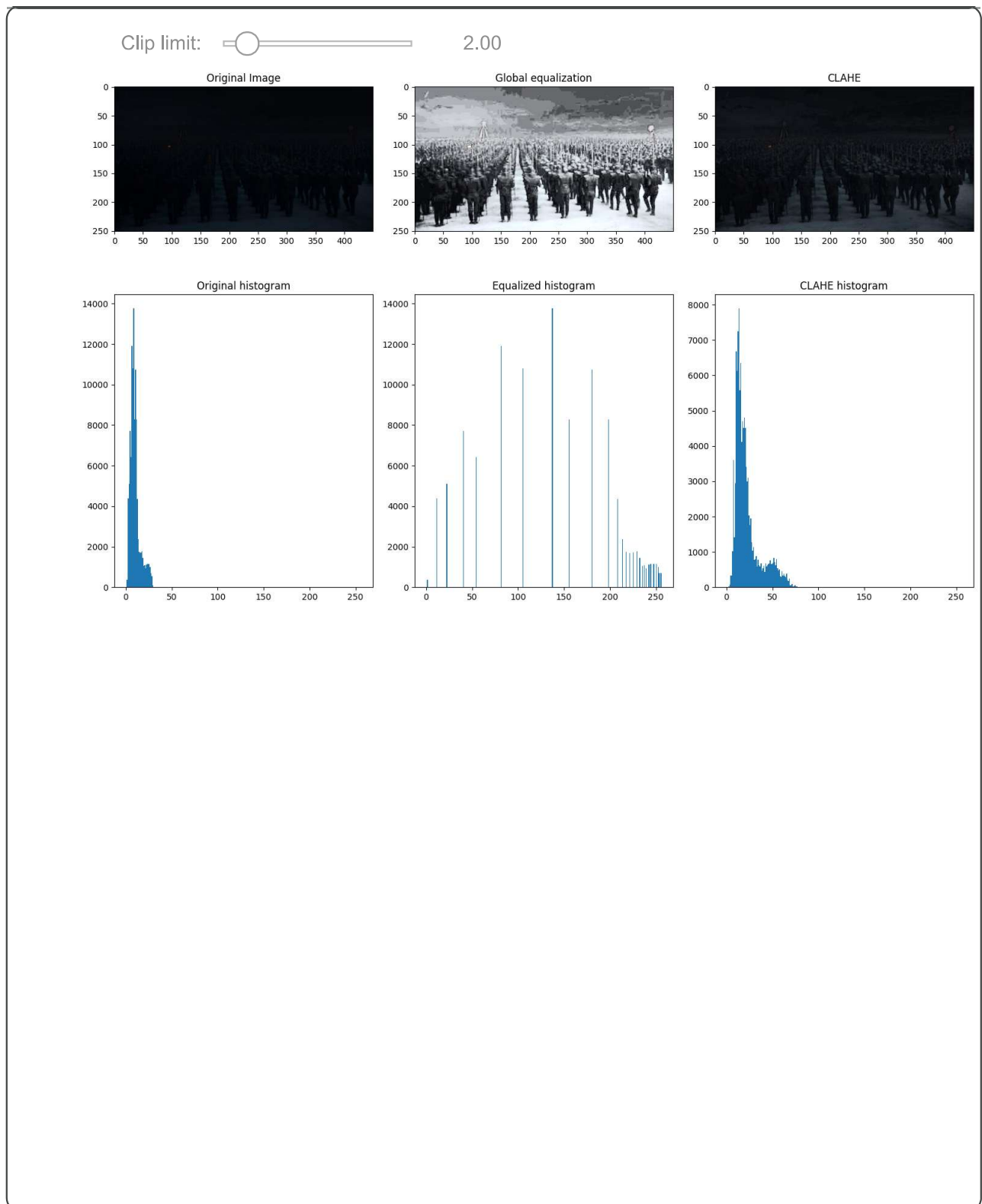
plt.tight_layout()
plt.show()

return out_image
```

```
#Create widget object
clip_widget = widgets.FloatSlider(value=2.0, min=1.0, max=10.0, step=0.5, description='Clip Limit')

# Read image
image = cv2.imread(images_path + 'batalla.jpg')

# Interact with your code!
interactive(clahe_chart, image=fixed(image), clip_limit=clip_widget, tile_grid_size=10)
```



Although we can see our enemies, the equalized image presents noise in the background and we can get a better quality picture with another image enhancement techniques, such as CLAHE(Contrast Limited Adaptive Histogram Equalization).Clahe will let us limit contrast enhancement and avoid noise amplification We are going to divide the image into small regions called tiles and we are going to equalize each tile locally using a **clipLimit** before calculating the cumulative distribution function (CDF).



CHAPTER II: THE EDGES OF THE LONG NIGHT

The ravens keep sending us images from beyond the Wall. However, due to the heavy fog and thick snow, our scouts cannot easily distinguish where the background ends and an enemy begins. To mount an effective watch from the towers of Winterfell, we need an edge detection system to help us recognize enemies, weapons, or incoming siege structures.

▼ DIRECTIONAL EDGE DETECTION (SOBEL)

For this technique, we are going to work with an image of Winterfell. Edges correspond to sharp transitions in pixel intensity. A difference must be significant enough to be considered a true boundary.

There are several edge detection methods, but we will start with first-derivative operators. By convolving the image with Sobel kernels (G_x and G_y), we can calculate the horizontal and vertical gradients. This allows us to estimate the boundary directions and combine them to compute the total gradient magnitude.

```
def edge_detection_chart(image, kernel_h, kernel_v, threshold, verbose=False):
    """ Computed the gradient of the image, binarizes and display it.

    Args:
        image: Input image
        kernel_h: kernel for horizontal derivative
        kernel_v: kernel for vertical derivative
        threshold: threshold value for binarization
        verbose: Only show images if this is True

    Returns:
        edges: edges binary image
    """
    # Compute derivatives
    d_h = cv2.filter2D(image, cv2.CV_16S, kernel_h)
    d_v = cv2.filter2D(image, cv2.CV_16S, kernel_v)

    # Compute gradient
    gradient_image = np.add(np.absolute(d_h), np.absolute(d_v))

    # Normalize gradient
    norm_gradient = cv2.normalize(gradient_image, None, 0, 255, cv2.NORM_MINMAX)

    # Threshold to get edges
    ret, edges = cv2.threshold(norm_gradient, threshold, 255, cv2.THRESH_BINARY)

    if verbose:
        # Set dark style and adjust figure size
        with plt.style.context('dark_background'):
            plt.figure(figsize=(15, 5))
```

```

# Show the initial image
plt.subplot(131)
plt.imshow(image, cmap='gray')
plt.title('Original image')
plt.axis('off')

# Show the gradient image
plt.subplot(132)
plt.imshow(gradient_image, cmap='gray')
plt.title('Gradient image')
plt.axis('off')

# Show edges image
plt.subplot(133)
plt.imshow(edges, cmap='gray')
plt.title('Edges detected')
plt.axis('off')

plt.tight_layout()
plt.show()

return edges

```

```

# Read image
image = cv2.imread(images_path + 'winterfell_blizzard.jpg', 0)

# Define kernel (Sobel)

kernel_h = np.array([[1,0,-1],[2,0,-2],[1,0,-1]])*1/4
kernel_v = np.array([[-1,-2,-1],[0,0,0],[1,2,1]])*1/4

interactive( edge_detection_chart, image=fixed(image), kernel_h=fixed(kernel_h), k

```



As we can see in the original image, the edges transition from dark pixels to brighter ones. This is the reason why we need to use a lower threshold. In this case, a value of

30 works well because it allows us to detect the outlines of the structures and terrain perfectly.

The Lookout's Dilemma: Single-Threshold Edge Detection

✓ CANNY EDGE DETECTION

To overcome the "Lookout's Dilemma" of using a single global threshold, we are going to use the Canny Edge detection, with two thresholds.

The Canny detector consists of four main steps:

1. **Noise Reduction:** Applies a 5x5 Gaussian filter to smooth the image and remove noise.
2. **Intensity Gradient:** Calculates the horizontal and vertical derivatives to find the gradient magnitude and direction.
3. **Non-Maximum Suppression:** Thins out the edges by suppressing pixels that are not local maxima in the direction of the gradient.
4. **Hysteresis Thresholding:** Uses two threshold values (`threshold1` and `threshold2`):
 - Pixels with a gradient intensity above `threshold2` are classified as strong edges (sure edges).
 - Pixels below `threshold1` are discarded.
 - Pixels between these two thresholds are only kept if they are connected to a strong edge.

```
def canny_edge_chart(image, threshold1, threshold2, verbose=False):
    """ Applies Canny edge detection and displays the result with a dark background """
    # Apply Canny algorithm
    edges = cv2.Canny(image, threshold1, threshold2)

    if verbose:
        with plt.style.context('dark_background'):
            plt.figure(figsize=(12, 6))

            plt.subplot(121)
            plt.imshow(image, cmap='gray')
            plt.title('Original Image')
            plt.axis('off')

            plt.subplot(122)
            plt.imshow(edges, cmap='gray')
            plt.title('Canny Edges (Hysteresis)')
            plt.axis('off')
```

```
plt.tight_layout()
plt.show()
```

```
# Read the same winterfell image
image_winterfell = cv2.imread(images_path + 'winterfell_blizzard.jpg', 0)

# Set up interactive sliders for both thresholds
interactive(canny_edge_chart,
            image=fixed(image_winterfell),
            threshold1=widgets.IntSlider(value=50, min=0, max=255, step=5, descrip
            threshold2=widgets.IntSlider(value=150, min=0, max=255, step=5, descri
            verbose=fixed(True))
```

Threshold ...  55